CSED233 · 데이터구조 · 포스텍

정리본 3. 알고리즘 분석

좋은 알고리즘의 조건 · 점근 분석 · Big-O/Ω/Θ · 최악/평균/최선.

1. 알고리즘과 성능

알고리즘(Algorithm): 문제를 유한한 시간 내에 해결하기 위한 단계별 절차.

예: 1부터 100까지의 합 — 입력 100 → 절차: sum에 1..100을 더함 → 출력 5050.

1.1 좋은 알고리즘의 5가지 조건

1. **입력(Input)**이 명확한가 — 배열, 정수, 문자열 등.
2. **출력(Output)**이 명확한가 — 정렬된 배열, 최댓값, true/false 등.
3. **명확성(Definiteness)** — "비슷한 것"이 아니라 "정확히 이것".
4. **유한성(Finiteness)** — 허용되는 모든 입력에 대해 알고리즘이 유한한 횟수의 단계를 실행하고 반드시 종료해야 한다. 반복문의 존재 자체가 문제가 아니라 종료 조건과 그 조건에 도달한다는 보장이 중요하다. 정상 종료 예: `for (int i = 0; i < n; ++i)` 는 정상적인 조건에서 유한하게 종료한다. `while (true) { break; }` 도 한 번 실행 후 종료한다. 종료하지 않는 예: `while (true) { }` 는 종료 조건이 없어 유한성을 만족하지 않는다.
5. **실행 가능성(Effectiveness, 유효성)** — 알고리즘의 각 연산은 명확하게 정의되어야 하며, 유한한 시간 안에 정확하게 수행할 수 있어야 한다. 실행 가능성은 연산 횟수나 메모리 사용량을 줄이는 효율성(Efficiency)과 구별된다.

1.2 성능은 두 축으로 측정한다

- **Time complexity(시간복잡도)** $T(n)$: 입력 크기 n에 따른 기본 연산 횟수의 증가 양상. 예: 선형 탐색은 최선 1회, 최악 n회의 원소 비교를 하므로 최악 시간복잡도는 O(n)이다.
- **Space complexity(공간복잡도)** $S(n)$: 입력 크기 n에 따른 메모리 사용량의 증가 양상. 32비트 정수 n개를 저장하는 배열의 원소 데이터는 4n bytes로 Θ(n)의 공간을 차지한다. 입력 자체의 저장 공간과 알고리즘 실행 중 추가로 필요한 보조 공간(auxiliary space)은 구별한다. 예를 들어 기존 배열을 순회하는 선형 탐색의 보조 공간은 Θ(1)이다.

- 실무에서 시간복잡도와 공간복잡도의 우선순위는 문제의 제약에 따라 달라진다. 응답 지연이 중요한 서비스에서는 실행 시간이 우선될 수 있고, 메모리 제한이 엄격한 임베디드 시스템이나 대용량 데이터 처리에서는 공간 사용량이 우선될 수 있다. 알고리즘 선택 시 입력 크기, 시간 제한, 메모리 제한을 함께 고려해야 한다.

2. 두 가지 분석 방법

	Empirical(경험적 분석)	Theoretical(이론적 분석)
방법	실제로 구현 → 실행 → 시간 측정	고수준 설계를 수학적으로 계산
장점	실제 성능 데이터 · 환경(CPU/OS) 반영 · 직관적	환경 독립 · 일반적 예측 · 큰 n도 계산 가능
단점	구현 비용 · 환경 따라 결과 변동 · 큰 n 측정 곤란	계산 모델과 실제 시스템의 차이로 실측과 괴리 가능. 이론적 분석은 T(n)=2n+3처럼 상수항·계수를 포함할 수 있으며, 상수 인자와 낮은 차수 항을 무시하는 것은 점근 표기법으로 성장률을 표현할 때의 특징이다

```
// 경험적 분석의 예: 직접 재 본다
#include <chrono>
#include <iostream>
auto start = std::chrono::steady_clock::now();
auto result = linearSearch(arr, target);
auto end = std::chrono::steady_clock::now();
double elapsed_ms = std::chrono::duration<double, std::milli>(end - start).count();
std::cout << "경과 시간: " << elapsed_ms << " ms\n";
// clock()을 쓰려면 CPU 시간(ms) = 1000.0 * (end - start) / CLOCKS_PER_SEC 로 환산한다.
```

3. 점근 분석(Asymptotic Analysis): 왜 최고차항만 보는가

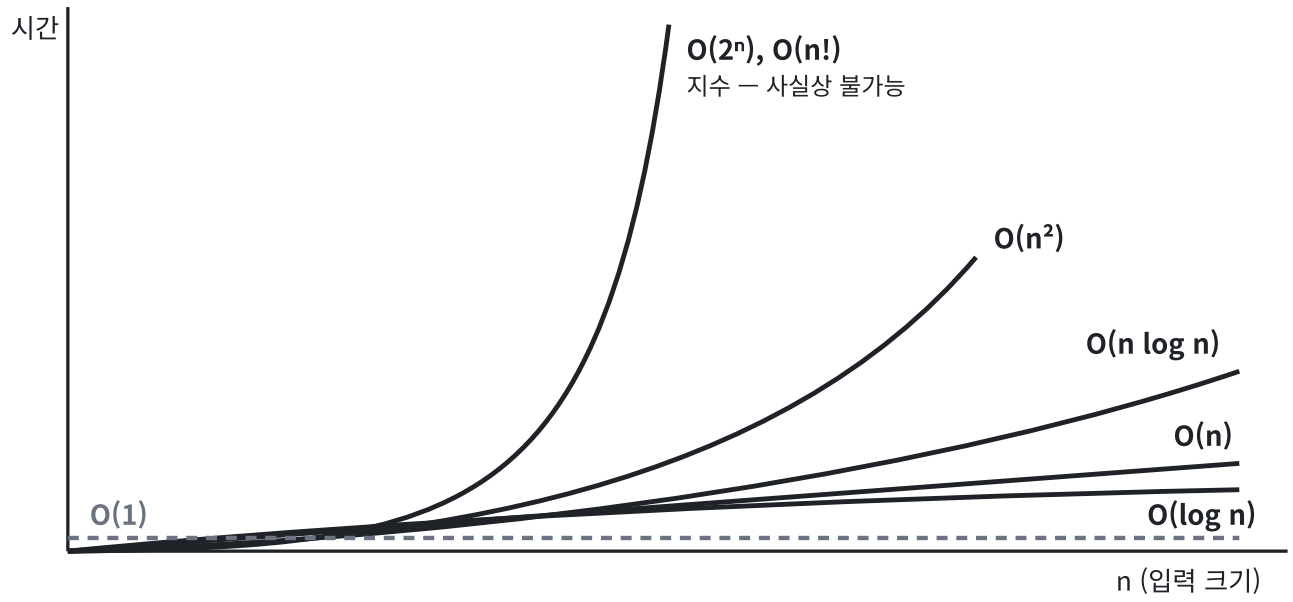
점근 분석은 n이 충분히 클 때의 극한 행동만 본다. 상수와 하위 항을 무시하고 지배 항만 남기는 이유를 수치로 확인해 보자.

$$T(n) = 2n^2 + 3n + 5$$

n	$2n^2$	3n	+5	T(n)	지배 항
1	2	3	5	10	상수항이 상대적으로 큼
10	200	30	5	235	n^2항이 지배적
100	20,000	300	5	20,305	n^2항이 거의 전부
1,000	2,000,000	3,000	5	2,003,005	최고차항 $2n^2$가 전체의 약 99.85%

→ n=1,000이면 $T(n)=2,003,005 \approx 2,000,000$으로, 최고차항 $2n^2$가 전체의 약 99.85%를 차지한다. n이 커질수록 최고차항의 상대적 비중이 증가한다. 따라서 $T(n)=2n^2+3n+5=\Theta(n^2)$이며, $O(n^2)$이기도 하다.

3.1 성장률 지형도



성장률 그래프는 상수 인자 등을 단순화한 대표 복잡도 함수들의 증가 추세를 비교한다. 입력 크기가 충분히 커질 때 일반적으로 $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$ 순으로 대표 함수의 성장 속도가 커진다. 단, Big-O는 상한을 나타내는 표기법이며 같은 n에서 실제 실행 시간의 순위를 보장하지 않는다. 상수 인자와 구현 비용이 다르면 작은 입력에서는 순위가 바뀔 수 있다. 대규모 입력에서는 성장률이 낮은 알고리즘이 일반적으로 유리하지만, 모든 문제에 $O(n \log n)$ 이하를 요구하는 것은 아니다.

3.2 n = 1,000일 때 대표 성장 함수 값 비교

실제 연산 횟수가 아니라, 각 복잡도 계열의 대표 함수 f(n)에 n=1,000을 대입한 값이다. 로그의 밑은 2로 둔다.

대표 함수 f(n)	f(1,000)	증가 양상
1	1	상수
$\log_2 n$	약 9.966	로그 증가
n	1,000	선형 증가
$n \log_2 n$	약 9,966	선형로그 증가
n^2	1,000,000	제곱 증가
n^3	1,000,000,000	세제곱 증가
2^n	약 1.072×10^{301}	지수 증가

주의: 대표 함수 값은 성장률을 비교하기 위한 기준값이다. 실제 알고리즘의 연산 횟수 또는 실행 시간이 이 값과 같다는 뜻은 아니다.

4. 점근 표기법 3종

Big-O — 상한 ("최대 이 정도")

$$T(n) = O(f(n)) \iff \exists\, c > 0,\ n_0 \geq 1 \text{ s.t. } T(n) \leq c \cdot f(n)\ \ (\forall n \geq n_0)$$

- 알고리즘이 아무리 느려도 c·f(n)을 넘지 않는다는 **위쪽 경계**.

Big-Ω — 하한 ("최소 이 정도")

$$T(n) = \Omega(f(n)) \iff \exists\, c > 0,\ n_0 \geq 1 \text{ s. t. } T(n) \geq c \cdot f(n)\ \ (\forall n \geq n_0)$$

- 예: 배열을 전부 봐야 하는 작업은 최소 $\Omega(n)$.

Big-Θ — 동등 ("정확히 이 정도")

$$T(n) = \Theta(f(n)) \iff \exists\, c_1, c_2 > 0,\ n_0 \geq 1 :\ c_1 f(n) \leq T(n) \leq c_2 f(n)\ \ (\forall n \geq n_0)$$

- 예: Merge Sort는 최악·평균 모두 $\Theta(n \log n)$.

예제 · Big-O 증명 연습

T(n) = 2n + 3 이 O(n)임을 보이려면 2n + 3 ≤ c·n을 만족하는 상수 c를 찾으면 된다.

$$2n + 3 \leq 3n \iff n \geq 3 \Rightarrow 2n + 3 = O(n)$$

c = 3으로 잡으면 n ≥ 3에서 성립 → O(n) ✓. 같은 식으로 $3n^2 + 5n + 7 = O(n^2)$, $5 = O(1)$, $2^n = O(2^n)$.

5. 최악 / 평균 / 최선

배열 [3, 1, 4, 8, 5, 9, 2, 6]에서 선형 탐색을 예로 들면:

경우	의미	선형 탐색 예	중요도
Worst(최악)	가장 오래 걸리는 입력	6 검색(끝에 있음) 또는 7(없음) → 8회 → O(n)	**가장 중요** — 성능 보장의 기준
Average(평균)	모든 입력의 평균 (찾는 대상이 반드시 존재하고, 8개 원소 각각이 동일한 확률로 검색 대상이 된다고 가정)	(1+2+⋯+8)/8 = 4.5회 → O(n). 일반 n에서는 (n+1)/2 = Θ(n) (실패 포함 시 입력 분포에 따라 달라짐)	현실적인 지표
Best(최선)	가장 빨리 끝나는 입력	3 검색 → 첫 원소에서 즉시 발견 → O(1)	최선 사례가 발생할 가능성은 실제 입력 분포에 달려 있다. 최선 분석은 알고리즘의 잠재적 효율성을 보여주지만 이것만으로 전체 입력에 대한 성능을 보장할 수는 없다

실무에서는 이렇게 고른다

- **최악 성능이 중요한 경우** — 실시간 처리가 필요한 시스템이나 엄격한 지연 제한이 있는 서비스에서는 최악 시간복잡도를 우선 분석한다. Merge Sort는 일반적인 비교 기반 구현에서 최악 O(n log n)의

시간복잡도를 보장하므로 정렬 알고리즘의 후보가 될 수 있다. 다만 실제 시간 제한 충족 여부는 최대 입력 크기, 구현 비용, 메모리 사용량 및 실행 환경을 별도로 검증해야 한다.

- **평균이 중요** — 일반 데이터 처리, 온라인 게임 → Quick Sort 평균 O(n log n).
- **최선이 중요** — 거의 정렬된 데이터 → Insertion Sort: 이미 정렬된 배열에서 최선 Θ(n), 역전 쌍의 개수가 I인 배열에서는 O(n+I)이라 역전 쌍이 적은 입력에서 특히 유리하다. 이는 최선 사례만 분석하는 것이 아니라 입력의 정렬 상태에 따른 적응적 성능을 활용하는 것이다.

6. Quiz

Q1. $O(n^2)$과 $O(n)$의 차이를 n = 1,000,000일 때 비교하라.

힌트

$n^2 = n \times n$. n에 10^6을 각각 대입해 보자.

정답

개념은 힌트 참고.

① $O(n)$: 10^6 = 100만 회 연산.

② $O(n^2)$: $(10^6)^2 = 10^{12}$ = 1조 회 연산.

③ 비율: $10^{12} / 10^6 = 10^6$.

정답: 100만 회 vs 1조 회 — 100만 배 차이.

주의: 이는 함수 n과 n^2를 직접 비교한 결과다. 실제 알고리즘의 연산 횟수나 실행 시간이 반드시 100만 배 차이 난다는 뜻은 아니다. $O(n)$, $O(n^2)$는 점근적 상한을 나타내므로 정확한 연산 횟수는 별도로 분석해야 한다.

Q2. O(n log n)은 어떤 의미인가?

힌트

$n \times \log_2(n)$. $\log_2(10^6) \approx 20$임을 이용해 $O(n^2)$과 비교해 보자.

정답

개념은 힌트 참고.

① $n = 10^6$이면 $\log_2(n) \approx 20$.

② $n \log_2(n) \approx 10^6 \times 20 = 2$천만 회.

③ $O(n^2) = 1$조 회와 비교하면 약 50,000배 빠름.

정답: 입력 n에 대해 대략 $n \times \log_2(n)$번의 연산. n = 100만이면 약 2천만 회로, $O(n^2)$보다 약 5만 배 빠르다.

이는 점근적 성장률을 이해하기 위한 대표 함수 비교이며, 실제 실행 시간이 5만 배 차이 난다는 보장은 아니다.

Q3. 왜 최악의 경우(Worst Case)를 기준으로 분석하는가?

힌트

"보장"의 관점에서 생각해 보자. 평균은 입력 분포에 따라 달라지지만 최악은 어떤 입력이 와도 성립한다.

정답

개념은 힌트 참고.

① 평균 분석은 입력 분포 가정이 필요하고, 가정이 틀리면 무너진다.

② 최악 분석은 어떤 입력이든 "이 이상은 안 걸린다"를 보장한다.

③ 실시간·금융처럼 지연이 곧 사고인 시스템에서는 이 보장이 필수다.

정답: 최악의 경우 분석은 크기 n인 허용 입력 중 최대 연산 횟수를 다루므로 입력 분포에 대한 가정 없이 성능 상한을 세울 수 있다. 단, 최악 시간복잡도 $T(n)=O(f(n))$은 충분히 큰 n에서 상수배의 점근적 상한이 존재한다는 뜻이지, 실제 경과 시간의 고정된 상한을 자동으로 보장하는 것은 아니다. 실시간 마감 시간을 보장하려면 시스템 환경과 최대 입력에 대한 추가 검증이 필요하다.

Q4. Bubble Sort $O(n^2)$와 Merge Sort $O(n \log n)$에서 n = 1,000일 때 차이는?

힌트

$\log_2(1000) \approx 10$. 각각 대입해 보자.

정답

개념은 힌트 참고.

① Bubble Sort: 대표 성장률 함수 $n^2 = 1000^2 = 100$만.

② Merge Sort: 대표 성장률 함수 $n \log_2 n \approx 1000 \times 10 =$ 약 1만.

③ 비율: 100만 / 1만 = 100배.

정답 — 대표 함수 비교: n=1,000일 때 n^2=1,000,000이고 $n \log_2 n \approx 9{,}966$이다. 대표 함수 값의 비율은 약 100배이다.

실제 최악 원소 비교 횟수: 표준 Bubble Sort는 499,500회, 일반적인 균형 분할 Merge Sort는 8,977회이다. 두 비교 횟수의 비율은 약 55.6배이다.

따라서 대표 함수의 비율, 실제 원소 비교 횟수의 비율, 실행 시간의 비율은 구별해야 한다.

Q5. 경험적 분석과 이론적 분석의 차이는?

힌트

측정이냐 계산이냐, 그리고 환경에 의존하느냐가 쟁점.

정답

개념은 힌트 참고.

① 경험적: 실제로 구현해 실행 시간을 측정 → 현실적이지만 환경(CPU/OS)에 따라 달라진다.

② 이론적: 수학적으로 연산 횟수를 계산 → 환경 독립적이고 일반화된다. 이론적 분석 자체가 상수를 반드시 무시하는 것은 아니며, 점근 분석과 구별해야 한다.

정답: 경험적 = 구현 후 측정(환경 의존), 이론적 = 수학적 계산(환경 독립, 일반적).